

Open Clinical Record

Technical Documentation

Architecture, Implementation, and Operations

Version 1.0 — Week 8 Finalization

27 September 2026

Repository	<code>elan-thinks/open-clinical-record</code>
Branch	<code>main</code>
Audience	Developers, technical mentors, evaluators
Companion	SRS v2.0, User Manual

Contents

1 Purpose and Scope	2
2 Architecture	2
2.1 Logical architecture	2
2.2 Design principles	2
2.3 Major components	2
3 Technology Stack	3
4 Repository Layout	3
5 Domain Model	3
5.1 Core relationships	3
5.2 Patient status	3
5.3 Visit status	4
5.4 Appointment status and transitions	4
6 Security and Authorization	4
6.1 Authentication	4
6.2 Authorization matrix (summary)	4
6.3 Secrets	5
7 API Surface (Illustrative)	5
8 Database and Migrations	5
9 Frontend Structure	5
10 Local Development	5
11 Testing and Continuous Integration	6
11.1 Automated tests	6
11.2 CI	6
11.3 Manual verification	6
12 Operational Notes	6
13 Known Limitations	7
14 Related Documents	7

1 Purpose and Scope

This document describes how Open Clinical Record (OCR) is designed and built. It supports maintenance, evaluation, and handover of the internship MVP.

In scope: patient management, longitudinal patient chart (visits), appointment management including check-in, authentication and RBAC, audit, testing, and local deployment.

Out of scope: pharmacy, laboratory, radiology, billing, FHIR exchange, multi-facility enterprise scheduling, and production multi-tenant SaaS concerns beyond what is needed for a demonstrable MVP.

2 Architecture

2.1 Logical architecture

```

React SPA (TypeScript, Vite)
  | HTTPS/HTTP JSON + Bearer JWT
ASP.NET Core 8 Web API
  | Application services
  | EF Core 8 + Npgsql
PostgreSQL

```

2.2 Design principles

- **Thin controllers**, rich application services for workflows and queries.
- **Authorization at the API**: policies and role claims; the UI only shapes experience.
- **Longitudinal clinical model**: Patient → ClinicalVisit → content. Prior visits are never overwritten.
- **Explicit state machines** for appointment and visit statuses.
- **Migrations as schema truth**: reference SQL is secondary documentation.

2.3 Major components

Component	Responsibility
Auth controller / JWT	Login, token issuance, role claims
Patient services	Register, search, update, deceased path
AppointmentWorkflowService	Status transitions, reschedule, cancel reason, check-in
ClinicalChartService	Chart reads/writes, visit documentation, finalize
DashboardService	Aggregate operational statistics
AuditService	Persist and query audit events
React pages + services	Role-aware UI and typed API clients

3 Technology Stack

Layer	Choice
Frontend	React 18, TypeScript, Vite
Backend	ASP.NET Core 8
ORM	Entity Framework Core 8
Database	PostgreSQL
Authentication	JWT Bearer
Tests	xUnit, WebApplicationFactory
CI	GitHub Actions (dotnet test; frontend build)

4 Repository Layout

```
open-clinical-record/
  src/backend/OpenClinicalRecord.Api/
  src/frontend/open-clinical-record-web/
  tests/backend/OpenClinicalRecord.Api.Tests/
  docs/
  .github/workflows/
```

Backend includes Controllers, Services, Data (DbContext + Migrations), Models/Entities, and DTOs. Frontend includes pages, services (API clients), layouts, routes with role guards, and context for session.

5 Domain Model

5.1 Core relationships

Patient (1)

```
|-- * Allergies, MedicalHistoryItems
|-- 0..1 DeathRecord (when Deceased)
|-- * Appointments
|-- * ClinicalVisits
    |-- 0..1 VitalSigns
    |-- * Diagnoses
    |-- * ClinicalNotes
```

Optional link: `ClinicalVisit.AppointmentId` connects check-in to the visit created for that attendance.

5.2 Patient status

Active, Inactive, Deceased. Deceased blocks new appointments and is backed by death-record provenance where recorded.

5.3 Visit status

- **Draft** — editable clinical documentation (default for new visits).
- **Final** — immutable; further care requires a new visit.
- **Cancelled** — voided visit.

Finalizing a visit that is linked to an appointment sets the appointment to **Completed** so the patient leaves the active queue.

5.4 Appointment status and transitions

Canonical statuses: Scheduled, Waiting, CheckedIn, InProgress, Completed, Cancelled, NoShow.

From	Allowed to
Scheduled	Waiting, CheckedIn, Cancelled, NoShow
Waiting	CheckedIn, Cancelled, NoShow, Scheduled
CheckedIn	InProgress, Waiting, Completed, Cancelled
InProgress	Completed, CheckedIn
Cancelled	/ Scheduled (re-open path)
NoShow	
Completed	(terminal)

Cancel requires a non-empty reason. Reschedule is allowed for Scheduled, Waiting, Cancelled, NoShow; blocked for CheckedIn, InProgress, Completed. Check-in creates a Draft visit when appropriate. AppointmentEvent rows preserve history.

6 Security and Authorization

6.1 Authentication

Users authenticate via login; the API issues a JWT including role claims. Subsequent requests send `Authorization: Bearer <token>`.

6.2 Authorization matrix (summary)

Action	Admin	Doctor	Nurse	Recep.
Register / search patients	Y	Y	Y	Y
Book appointment	Y	Y	Y	Y
Clinical chart write	N	Y	Y	N
Mark deceased	Y	Y	N	Y
Clear deceased	Y	Y	N	N
Audit / users	Y	N	N	N

Policies such as `ClinicalStaff`, `StaffCanBook`, `CanMarkDeceased`, `CanClearDeceased`, and `AdminOnly` encode these rules. Full detail lives in `docs/05-engineering/access-control-report.md`.

6.3 Secrets

Connection strings and JWT signing keys must come from configuration and environment variables, not committed secrets.

7 API Surface (Illustrative)

Area	Examples
Auth	POST /api/auth/login
Patients	GET/POST /api/patients, deceased endpoints
Appointments	GET/POST /api/appointments, PATCH .../status, PATCH .../reschedule
Chart	GET/POST/PATCH .../patients/{id}/chart/...
Dashboard	GET /api/dashboard/stats
Audit	GET /api/audit (Admin)
Health	GET /api/health, /api/health/ready

8 Database and Migrations

- Authoritative schema: `src/backend/OpenClinicalRecord.Api/Migrations/`
- Reference SQL: `docs/05-data/ocr-complete-database.sql` (may lag; do not prefer over migrations)
- Apply with `dotnet ef database update` (or startup migration if configured)

Historical clinical rows use restrictive delete behaviour where appropriate so patient deletion cannot casually erase clinical meaning. Appointment events cascade with appointments. Visit content cascades with the visit. Optional appointment links on visits use SET NULL so removing an appointment does not erase the visit record.

9 Frontend Structure

- `src/pages` — route-level screens
- `src/services` — API clients and error types
- `src/routes` — route table and role guards
- `src/layouts` — application shell
- `src/context` — authentication session

API base URL is configured via `VITE_API_BASE_URL` (default local API port as documented in the project).

10 Local Development

```
# Backend
cd src/backend/OpenClinicalRecord.Api
dotnet run --launch-profile http

# Frontend
cd src/frontend/open-clinical-record-web
npm install
npm run dev
```

Typical local URLs: frontend `http://localhost:5173`, API `http://localhost:5000`. Development CORS allows the Vite origin.

11 Testing and Continuous Integration

11.1 Automated tests

Backend integration tests cover:

- Access matrix (role permissions)
- Appointment workflow (transitions, cancel reason, reschedule, check-in Draft visit)
- Clinical documentation and finalize (including InMemory-safe patterns)
- Longitudinal visits (history not overwritten)

```
cd tests/backend/OpenClinicalRecord.Api.Tests
dotnet test
```

11.2 CI

GitHub Actions runs backend tests and frontend production build (TypeScript check + Vite build) on pushes to `main`.

11.3 Manual verification

End-to-end checklist: `docs/09-testing/e2e-clinical-flow-checklist.md`.

12 Operational Notes

- Prefer environment-based configuration for JWT key length and database credentials.
- Training seed users exist only for development demonstration.
- Health endpoints support basic liveness and readiness checks.
- Backup and restore of PostgreSQL should be defined before any production-like deployment.

13 Known Limitations

- Reference SQL dump may lag the latest EF migrations.
- Reports are basic aggregates, not a full analytics platform.
- No multi-tenant or multi-facility model.
- Frontend automated UI tests are limited relative to backend API tests.

14 Related Documents

- SRS v2.0 — formal requirements and acceptance criteria
- User Manual — role-based operational guide
- Clinical domain rules — status machines and business rules
- Access-control report — live permission matrix and seed accounts
- Project structure — repository map

Document control

Technical Documentation version 1.0 (27 September 2026). Implementation evidence is the **main** branch of the project repository.